Shared Element Primer

Kairos uses an architecture formed around a defined set of shared memory elements. This shared memory is common between computing elements distributed throughout Kairos Autonomi's robotic systems. The shared memory is discretized into usable elements called shared variables. Shared Variables, or SVs, contain data related to specific functions. Each shared variable has a source and a user. I most cases there is a single source and multiple users for each shared variable.

The goal of this document is to provide an overview of the SharedLInk protocol. A document called the SharedLink Teleop Only ICD provides the detail of SharedLink implementation.

Although named pipes are an important part of the Kairos robotic architecture, they are not discussed or addressed in this document.

Shared Variables Data Dictionary

The instantiation of shared variables from a pool of shared memory creates a data dictionary. Each shared variable has at a minimum a name, a type and a handle. It can further have change flags, access counters, semaphores, units and many other attributes. The primary data dictionary has at a minimum a name, a type and a handle per implemented system. Variables defined in the data dictionary may not be created in a specific order so the SV 'Handle' can be different from app to app for the same shared variable. The 'Handle' is simply the sequence number assigned when the SV was created in the system. The most universal way to reference a shared variable is through its case sensitive 'Name'. It is possible to 'compile' the handles of the used SVs at runtime into handle reference tabled for significant improvements in processing speed.

Shared Variable Sources

A shared variable can receive its data from a number of sources, but generally one at a time. The SV can be filled as a constant, as a user input or translated from a streaming protocol such as NMEA GPS velocity. The SV source performes the necessary conversions required and then in either a synchronous or asynchronous fashion, writes the value into the shared variable.

Shared Variable User

A shared variable can be used by any application in the system capable of reading the shared memory pool. It can be read multiple times by different applications. They can be referenced by name or handle. The values are atomized at a system level so they can be read and written simultaneously. Users can read the shared variable synchronously or asynchronously based upon the requirement of the application using the shared variable.

Shared Variable transmission Protocol - SharedLink

Two pools of SVs located on two separate computer systems can be synchronized such as a distant robot and its OCU. A protocol called SharedLink is used to provide that synchronization. SharedLink is a directional protocol that consists of two lists sets. Each list set has an inbound and an outbound component. The inbound list receives SV data from the outbound list of a different computer system. The converse is also true.

The SharedLink protocol connects a group of shared variables together between two computer system. The protocol consists of identical Inbound and Outbound lists located on each computer. These list are linked between computer systems, meaning that changes in shared variables on the computer with the OutBound list are reflected as changes in the shared variables on the computer with the Inbound list. The Inbound List is the shared variable 'Source' for that computer systems. The Outbound shared variable list is just one of the uses from that computer system.

SharedLink consists of a packet that contains functional fields. These fields perform operations on the Shared Variable pool of the receiving computer.

SharedLink transmissions consist of a continuous stream of packets between the two computers systems. Data compression is used to reduce the packet size while maintaining synchronization between the local inbound and remote outbound lists for each computer. Data compression occurs when the data does not change between packets.

Actions between computer systems are based upon the changes in value of a Shared Variable and not the actual SharedLink messages. SharedLink simply transports the value of a Shared Variable as it changes over time.

The synchronization between the local Outbound list and the remote Inbound list is continuous. SharedLink packets are continuously transmitted at a set rate of nominally 100ms. These packets contain data place holders if no change has taken place or data if there has been a change. Once per second all the data is sent independent of change to assure shared variable synchronization. The nominal rate of 100ms can be changed on a system by system based upon performance requirements.

This approach works over a serial port or Ethernet as UDP messages. Each packet has its own checksum and addressing. Packets are accepted if the addressing and checksum are correct or ignored is they are incorrect. Since the protocol is streaming, individual packet loss is of low consequence.

The following datagrams show how the SharedLink process takes place.

Let us assume for the sake of discussion that there is an Outbound List on an OCU computer with three shared variables in it with initial values

Veh_Brake	100	-Values written on the local computer
Veh_Throttle	0	Read by SharedLink, then transmitted
Veh_Steer	15	

It will be targeted at an Inbound list on a remote computer in a robot.

Veh_Brake	100	-Values read by the remote computer
Veh_Throttle	0	after they are written to by SharedLink
Veh_Steer	15	

This SharedLink packet consists of the data for an Outbound list of 3 shared variables. It is called a 'full' packet because each of the shared variables contain data. This packet is sent on a once per second basis as synchronization or because all of the data in each shared variable has changed since the last tick time. Of 100ms or so.

[:BA|N0|D100|D0|D15|CXX]

The SharedLink message where there was no change in any shared variable in the Outbound list. The Shared variable list consists to 3 shared variables. Note that there is no data supplied after the field identifier D because there was no change since the last SharedLink packet was sent.

[:BA|N0|D|D|D|CXX]

SharedLink message where there was there was a single change in the 1st shared variable in the Outbound list. Note that there is data supplied only after the 1st data field that is related to the 1st entry in the Outbound SV list

[:BA|N0|D0|D|D|CXX]

SharedLink message where there was there was a single change in the 2nd shared variable in the Outbound list. Note that there is data supplied only after the 2nd data field that is related to the 2nd entry in the Outbound SV list

[:BA|N0|D|D75|D|CXX]

SharedLink message sent as synchronization between the local and remote computers. This is sent on a once per second basis where change data is sent on change or at a nominal 100ms rate.

[:BA|N0|D100|D75|D15|CXX]

The previous datagrams take place over a period of time. During that time many packets will be transmitted as synchronization is assured between systems. The sequence and the quantities of the packets may change based upon actual system function. Below is an example of what the data traffic may actually look like as it is viewed in realtime.

```
[:BA|N0|D100|D0|D15|CXX]      // 1 second Synchronization
[:BA|N0|D|D|D|CXX]            //No change in list packets
[:BA|N0|D|D|D|CXX]
[:BA|N0|D|D|D|CXX]
[:BA|N0|D|D|D|CXX]
[:BA|N0|D|D|D|CXX]
[:BA|N0|D|D|D|CXX]            //No change in list packets
[:BA|N0|D|D|D|CXX]
[:BA|N0|D|D|D|CXX]
[:BA|N0|D|D|D|CXX]
[:BA|N0|D100|D0|D15|CXX]      // 1 second Synchronization
[:BA|N0|D|D|D|CXX]            //No change in list packets
[:BA|N0|D|D|D|CXX]
[:BA|N0|D0|D|D|CXX]           //1st shared Variable changed
[:BA|N0|D|D|D|CXX]
[:BA|N0|D|D75|D|CXX]          // 2nd Shared variable changed
[:BA|N0|D|D|D|CXX]            //No change in list packets
[:BA|N0|D|D|D|CXX]
```

[:BA|N0|D|D|D|CXX]
[:BA|N0|D|D|D|CXX]
[:BA|N0|D0|D75|D15|CXX] // 1 second Synchronization

Remember that the quantity of the packets is not what is controlling the values of the shared variables, but the change packets and the synchronization packets. This approach is used because of the 'loss' nature of communications protocols. Loss of singular packets has low impact on the entire system.

System Example

A Shared Variable called Veh_Brake is used to contain the desired position of the service brake on a remote robotic system. The current value in Veh_Brake is 0. The local OCU process can write to the SV called Veh_Brake a value of 75 as its source. Veh_Brake is in the Outbound list of the local computer and Veh_Brake is in the Inbound list of the remote robotic system.

The local SharedLink process scans the Shared Variables in the Outbound list for a change at a rate of, say 100ms. If a change is detected then the Shared link protocol places that change at the location in the SharedLink packet specified by its location in the Outbound list and sends the packet to the remote robotic system.

The remote robotic system receives the SharedLink packet, decodes it based upon the sequence of shared variables as defined by the Inbound shared variables list. The value is then stored in the sahred variable as its source.

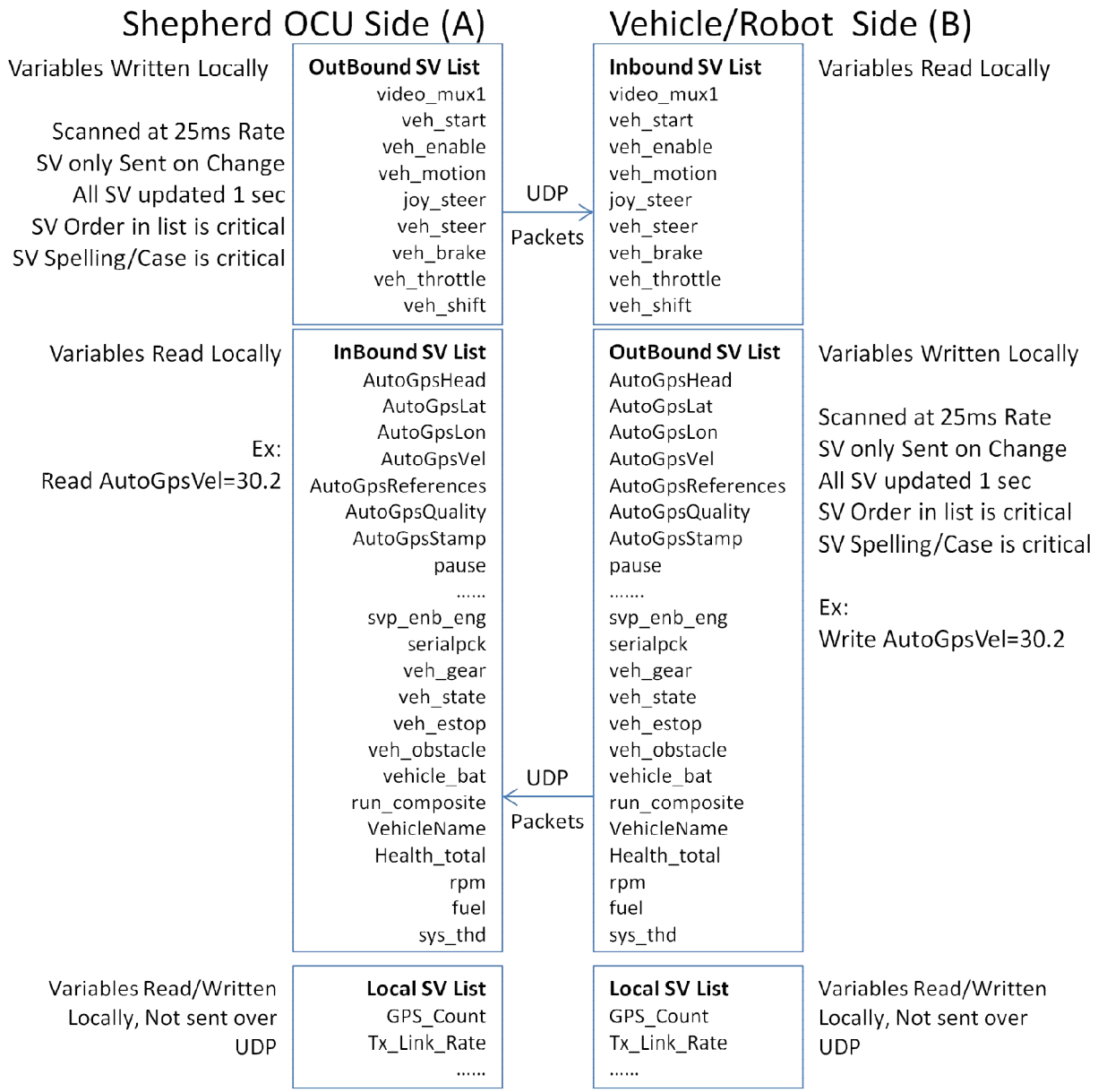
The remote robotic system can then use the value of the Veh_Brake shred variable in the system, presumably to adjust the position of the vehicle brake actuator.

Using this approach it is possible to create and control large numbers of shared variables across a communications channel.

SharedLink has the ability to specifically adjust the value of singular shared variables along with many other features.

Full Shared Link System

This diagram depicts a full instantiation of a SharedLink protocol between two computer systems. It shows fully developed OutBound and Inbound SV lists and their connection between the systems.



The left side is a OCU and the Right side is the remote robotic systems that it controls. Changes made in Shared Variables on the Outbound list of the left side also occur on the Inbound list of right side because of the SharedLink protocol. The transmissions and list synchronizations occur both left to right and right to left.